

```
# Test code
print(switchIsOn)
turnOn()
print(switchIsOn)
turnOff()
print(switchIsOn)
turnOn()
print(switchIsOn)
```

Listing 2-1: Model of a light switch written with procedural code

The switch can only be in one of two positions: on or off. To model the state, we only need a single Boolean variable. We name this variable `switchIsOn` ❸ and we say that `True` means on and `False` indicates off. When the switch comes from the factory, it is in the off position, so we initially set `switchIsOn` to `False`.

Next, we look at the behavior. This switch can only perform two actions: “turn on” and “turn off.” We therefore build two functions, `turnOn()` ❶ and `turnOff()` ❷, which set the value of the single Boolean variable to `True` and `False`, respectively.

I’ve added some test code at the end to turn the switch on and off a few times. The output is exactly what we would expect:

```
False
True
False
True
```

This is an extremely simple example, but starting with small functions like these makes the transition to an OOP approach easier. As I explained in Chapter 1, because we’ve used a global variable to represent the state (in this case, the variable `switchIsOn`), this code will only work for a single light switch, but one of the main goals of writing functions is to make reusable code. I’ll therefore rebuild the light switch code using object-oriented programming, but I need to work through a bit of the underlying theory first.